can grow into a GPL before you even realize it. At that point, your leisurely foray into language writing will have escalated into an exhausting marathon.

To avoid these difficulties, you can pick a different route. Rather than writing a full-fledged DSL, you can bend a GPL into something resembling a DSL for your specific problem. The next section shows you how.

Internal and External DSLs

Let's see an example of a DSL that's actually a GPL in disguise. Here's a snippet of Ruby code that uses the Markaby gem to generate HTML:

dsl/markaby_example.rb

```
require 'markaby'

html = Markaby::Builder.new do
  head { title "My wonderful home page" }
  body do
    h1 "Welcome to my home page!"
    b "My hobbies:"
    ul do
      li "Juggling"
      li "Knitting"
      li "Metaprogramming"
    end
  end
end
```

This code is plain old Ruby, but it looks like a specific language for HTML generation. You can call Markaby an *internal DSL*, because it lives within a larger, general-purpose language. By contrast, languages that have their own parser, such as make, are often called *external DSLs*. One example of an external DSL is the Ant build language. Even though the Ant interpreter is written in Java, the Ant language is completely different from Java.

Let's leave the GPL vs. DSL match behind us and assume that you want to use a DSL. Which DSL should you prefer? An internal DSL or an external DSL?

One advantage of an internal DSL is that you can easily fall back on the underlying GPL whenever you need to do so. However, the syntax of your internal DSL will be constrained by the syntax of the GPL behind it. This is a big problem with some languages. For example, you can write an internal DSL in Java, but the result is probably still going to look pretty much like Java. But with Ruby, you can write an internal DSL that looks more like an *ad hoc* language tailored to the problem at hand. Thanks to Ruby's flexible,